Serenisun Capstone: Planning Document

Project Idea

Serenisun is a yoga and meditation platform where users book live sessions with real instructors and get a personalized AI practice guide in between. Users browse services such as group yoga classes over Zoom, group guided meditations, and 1:1 private sessions, pick a date and time, reserve a spot, and manage their bookings from a personal dashboard. An integrated AI guide recommends the right session based on the user's experience and goals, and writes a short pre-session summary for the instructor. The app is built with React, Node.js/Express, and MongoDB.

Problem Statement

Beginners often don't know which yoga or meditation session suits them, and they find it hard to keep a practice going between classes. Picking the wrong class is discouraging, and many people drop off after a few sessions because they have no guidance at home.

Instructors, in turn, spend time answering "which class should I join?" and meet new students without knowing their experience or goals. Serenisun addresses both sides by matching users to the right session, giving instructors context beforehand, and supporting a consistent home practice that adjusts to how the user feels.

Target Users

- **Primary:** beginner and intermediate practitioners, especially busy people and students who want a regular practice but aren't sure where to start.
- **Secondary:** yoga and meditation instructors or small studios who want better-matched students and easier class management.

Scope

The AI layers are built in order so the lowest can be cut if time runs short.

Core features

- Registration, login, logout, and protected pages
- Service browsing with category, duration, and description, plus a detail page
- Booking with date and time, plus view, update, and cancel
- Capacity logic for group classes: each class has a maximum size, shows spots remaining, and blocks booking when full
- Double-booking prevention for 1:1 sessions
- Dashboard with upcoming bookings, Zoom links, and AI summaries
- Responsive design, validation, and error handling

- Public deployment, GitHub repo with 15+ commits, README, and documentation file

Services (seeded in the database)

Online sessions store a meeting link as a field, so there is no Zoom integration to build.

Service	Type	Duration (min)
Group Yoga Class	Group, online	60
Group Guided Meditation	Group, online	30
Private Yoga Session	1:1	60
Private Meditation Coaching	1:1	30
Intro Consultation	1:1	20

AI layers (in build order)

1. **Session recommendation and instructor summary.** The user describes their experience and goals, and the AI suggests a service, explains why, and writes a short note for the instructor. This alone satisfies the AI requirement.
2. **Personalized home practice (not built; future work).** From a short intake (experience, time available, goal such as relaxation, flexibility, or focus), the AI generates a yoga flow or a guided meditation script.
3. **Feedback loop (not built; future work).** After a practice, the user rates how it felt (difficulty, enjoyment, how calm they feel), and the AI adapts the next one.
4. **Progress tracking.** Practice streaks, minutes practiced, and milestones on the dashboard.

Data (MongoDB)

Users (including a practice profile), services, instructors, sessions or time slots (with capacity and attendees), bookings, generated practices, and feedback entries.

Stretch goals

- Read meditation scripts aloud with the browser's built-in text-to-speech
- Voice input for the intake form
- An instructor view of incoming bookings
- Email confirmations

Out of scope

Payments, real Zoom integration, video or image content, wearable data, and native mobile apps. For the demo, seed one or two fictional instructors and a set of time slots.

AI Boundaries

Because the scope is only yoga and meditation, the guardrails stay short:

- The AI does not give medical, nutrition, weight, or mental health advice, and does not diagnose or treat anything.
- Yoga flows stay gentle and beginner-safe, with a standing reminder to move within a comfortable range and stop if something hurts.
- If a user mentions an injury, a medical condition, pregnancy, or serious distress, the AI responds with a brief note to consult a doctor or talk to their instructor, and does not try to adapt around it.
- Meditation is presented as relaxation and focus practice, not therapy.

User Stories

ID	As a...	I want to...	So that...	Priority
US-1	New user	register with my email and a password	I can book sessions and keep my practice profile	High
US-2	Returning user	log in and log out securely	my bookings and data stay private	High
US-3	User	browse services filtered by category, type (group or 1:1), and duration	I can find a session that fits my time and goals	High
US-4	User	open a service detail page with its description, duration, and instructor	I know what to expect before booking	High
US-5	User	see available dates and times, and spots left in group classes	I can pick a session that is actually open	High
US-6	User	book a session by choosing a date and time	my spot is reserved	High
US-7	User	be blocked from booking a full class or an already taken 1:1 slot	I never end up with a booking that does not exist	High
US-8	User	view my upcoming and past bookings	I can keep track of my sessions	High
US-9	User	reschedule or cancel a booking	I can adjust when my plans change	High
US-10	User	see a dashboard with upcoming sessions, Zoom links, and AI summaries	I can find everything for my next session in one place	High
US-11	Beginner	describe my experience and goals in my own words and get a recommended service with a reason	I choose the right session the first time	High

ID	As a...	I want to...	So that...	Priority
US-12	User	have the AI write a short note for the instructor from my description	my instructor knows my level and goals before we meet	High
US-13	User	be told to consult a doctor or my instructor when I mention an injury, medical condition, pregnancy, or distress	the AI never gives me medical or mental health advice	High
US-14	Instructor	see my upcoming bookings with each student's AI-written note	I can prepare for the session	High
US-15	User	fill in a short intake (experience, time available, goal) and get a personalized yoga flow or meditation script	I can practice at home between classes	Low
US-16	User	rate a practice (difficulty, enjoyment, how calm I feel) and get an adapted next one	my home practice keeps fitting me	Low
US-17	User	see my practice streak, minutes practiced, and milestones	I stay motivated to keep going	Low
US-18	User	have a meditation script read aloud	I can close my eyes and follow along	Low
US-19	User	receive an email confirmation after booking	I have a record of the session outside the app	Low

Functional Requirements

Each requirement is something the system must do, and each traces back to one or more user stories.

ID	Area	The system shall...	Stories
FR-1	Auth	let a visitor register with name, email, and password, and reject duplicate emails	US-1
FR-2	Auth	log a user in with email and password and issue a token for later requests	US-2
FR-3	Auth	log a user out and block protected pages and API routes for anyone not logged in	US-2
FR-4	Auth	distinguish regular users from instructors, and limit instructor views to instructors	US-14
FR-5	Services	list all services with category, type (group or 1:1), duration, and description	US-3
FR-6	Services	filter the service list by category, type, and duration	US-3

ID	Area	The system shall...	Stories
FR-7	Services	show a detail page for each service with its instructor and meeting details	US-4
FR-8	Booking	show open time slots for a service, including spots remaining for group classes	US-5
FR-9	Booking	create a booking for a chosen service, date, and time for the logged-in user	US-6
FR-10	Booking	reject a booking when a group class is at capacity or a 1:1 slot is taken, and return a clear message	US-7
FR-11	Booking	prevent the same user from booking the same slot twice	US-7
FR-12	Booking	list a user's upcoming and past bookings	US-8
FR-13	Booking	let a user reschedule or cancel their own booking, and free the spot when it is cancelled	US-9
FR-14	Dashboard	show upcoming sessions, Zoom links for online sessions, and each booking's AI summary	US-10
FR-15	AI	accept a free-text description of experience and goals and return a recommended service with a short explanation	US-11
FR-16	AI	generate an instructor note from that description and attach it to the booking	US-12
FR-17	AI	detect mentions of injury, medical conditions, pregnancy, or distress and respond with a refer-to-a-professional message instead of advice	US-13
FR-18	AI	show a short disclaimer that the AI offers relaxation and practice guidance, not medical advice	US-13
FR-19	Instructor	show an instructor their upcoming bookings with student names and AI notes	US-14
FR-20	AI	accept a practice intake (experience, time available, goal) and generate a yoga flow or meditation script	US-15
FR-21	AI	save each generated practice and let the user rate it	US-15, US-16
FR-22	AI	use a user's saved ratings to adapt the next generated practice	US-16
FR-23	Progress	calculate and show practice streak, total minutes, and milestones	US-17
FR-24	Stretch	read a meditation script aloud using the browser's text-to-speech	US-18
FR-25	Stretch	send an email confirmation after a booking is created	US-19

ID	Area	The system shall...	Stories
FR-26	General	validate all input on the server and return consistent, readable error messages	All
FR-27	General	show loading, empty, and error states for every page that loads data or calls the AI	All

FR-1 to FR-19 and FR-26 to FR-27 are required for the core build. FR-20 to FR-22 are the second and third AI layers, FR-23 is the fourth, and FR-24 and FR-25 are stretch goals.

Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Pages load in under 3 seconds on a normal broadband connection, and non-AI API calls respond in under 500 ms
NFR-2	Performance	AI responses appear within about 10 seconds, with a visible loading indicator while the user waits
NFR-3	Security	Passwords are hashed (for example with bcrypt), never stored or logged in plain text
NFR-4	Security	Protected routes verify the user's token on the server, and users can only read or change their own bookings
NFR-5	Security	The AI API key and database credentials live in server-side environment variables, never in the frontend or the repository
NFR-6	Security	The app is served over HTTPS in deployment, and the server limits request rates, with a tighter limit on AI endpoints to control cost
NFR-7	Reliability	Booking is safe against two people grabbing the last spot at once, so capacity is never exceeded
NFR-8	Reliability	If the AI call fails or times out, the user sees a friendly message and can still book without it
NFR-9	Usability	The layout works from phone width (about 360 px) to desktop (1440 px), with clear navigation and consistent styling
NFR-10	Usability	Forms have labels, readable contrast, and keyboard-friendly controls, aiming at WCAG AA basics
NFR-11	Privacy	The app stores only what it needs (account details, bookings, practice ratings) and does not collect health or medical information
NFR-12	Maintainability	The backend separates routes, controllers, services, and models, and the frontend separates pages, components, and API calls, with no monolithic files

ID	Category	Requirement
NFR-13	Maintainability	Code uses meaningful names, a linter for consistent style, and comments only where the reason is not obvious
NFR-14	Testability	Main workflows (register, log in, book, cancel, AI recommendation) are tested end to end, and key API routes are checked with manual API tests (curl)
NFR-15	Deployability	The app deploys from the GitHub repository using documented steps: frontend on Netlify or Vercel, backend on Render or Heroku, database on MongoDB Atlas
NFR-16	Compatibility	The app works in current versions of Chrome, Firefox, Safari, and Edge, and speech features degrade gracefully where unsupported

Brand and Visual Direction

- **Name:** Serenisun, a blend of serenity and sun.
- **Tagline:** Expand your inner radiance.
- **Theme:** Organic futurism, which pairs soft, natural, flowing forms with a clean, modern, forward-looking interface.

Design principles

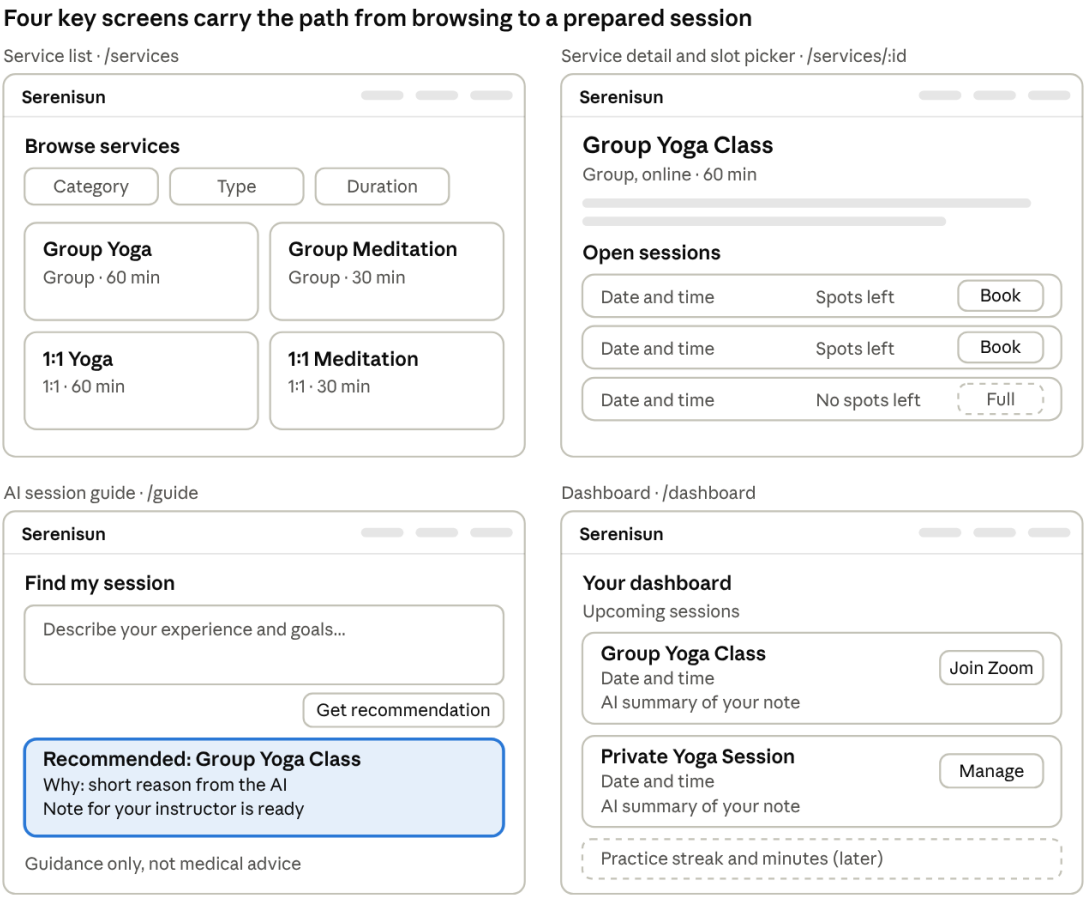
- **Shapes:** rounded corners, pill-shaped buttons, and soft curved dividers between sections, with no hard edges.
- **Imagery:** abstract sun glows, soft gradients, and plant-like line art instead of stock photos, since photo and video content is out of scope.
- **Typography:** a geometric or rounded sans for headings (for example Outfit or Poppins) and a readable humanist sans for body text (for example Nunito or Inter), all free on Google Fonts.
- **Motion:** slow, gentle transitions, with a soft pulsing glow while the AI guide is thinking. All motion is turned off for users who prefer reduced motion.
- **AI voice:** calm, warm, and brief, in line with the AI boundaries above.

Proposed palette

Role	Color	Hex
Sunlight (accent)	Warm gold	#F2B84B
Glow (secondary accent)	Soft coral	#F28C6B
Calm (supporting)	Sage green	#A8BFA3
Background	Warm cream	#FBF6EC
Surface	Pale mist	#E7EEE9
Text	Deep forest	#1F3B34

Text and button colors should be checked for contrast so they meet the readability goal in NFR-10.

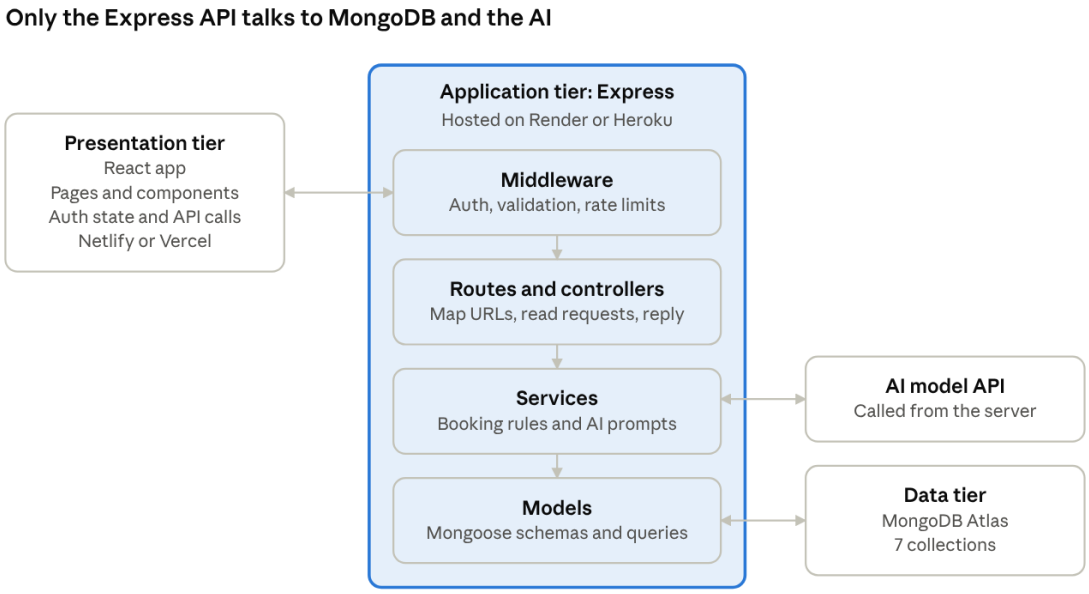
Wireframes



wireframes · 4 key screens

These low-fidelity wireframes cover the path from browsing to a booked, prepared session. Labels are shortened, and dates, times, and spot counts are placeholders. The full row on the detail page shows the capacity rule (FR-10), and the dashed box on the dashboard marks a later feature (US-17).

Architecture Overview



architecture · 3 tiers and an AI API

The React app, hosted on Netlify or Vercel, sends REST requests to the Express API. Each request passes through middleware, then routes and controllers, services, and models. Only the services call the AI, and only the models touch MongoDB Atlas, so the AI key and database credentials never reach the browser.

Database Schema

The database has 7 MongoDB collections, modeled with Mongoose. Services describe what is offered, sessions are the bookable time slots, and bookings connect users to sessions.

Collection	Key fields	Purpose
users	name, email (unique), passwordHash, role (user or instructor), practiceProfile (experience level, goals), createdAt	Accounts and each person's practice profile
instructors	userId (ref users), bio, specialties	Public profile for each instructor
services	name, category (yoga or meditation), type (group or 1:1), durationMinutes, description, isOnline	The catalog users browse
sessions	serviceId (ref), instructorId (ref), startsAt, endsAt, capacity, bookedCount, meetingLink	One bookable time slot of a service, with its spots

Collection	Key fields	Purpose
bookings	userId (ref), sessionId (ref), status (confirmed or cancelled), userDescription, aiRecommendation, instructorNote, createdAt	A user's reservation, plus the AI output attached to it
practices	userId (ref), type (yoga or meditation), intake (experience, minutes, goal), content, createdAt	A generated home practice
feedback	practiceId (ref), userId (ref), difficulty, enjoyment, calm (each rated 1 to 5), createdAt	Ratings used to adapt the next practice

Relationships

- One service has many sessions, and one instructor has many sessions.
- One session has many bookings, and one user has many bookings.
- One user has many practices, and each practice can have feedback.

Design decisions

- **Group and 1:1 share one structure.** A 1:1 session is a session with a capacity of 1, so the same booking logic enforces both capacity and double-booking prevention.
- **Capacity is enforced in the database.** Booking increments bookedCount only if it is still below capacity, in a single atomic update, so two people cannot take the last spot (NFR-7).
- **A unique index on user and session** stops the same user from booking a slot twice (FR-11).
- **AI output lives with the thing it describes.** The recommendation and instructor note are stored on the booking, so the dashboard can show them without calling the AI again.
- **Progress is calculated, not stored.** Streaks and minutes practiced are derived from practices and feedback (FR-23), which avoids numbers going out of sync.

Pages and Components

The React app has 11 pages built from a shared set of components. The backend mirrors the same separation of concerns.

Pages

Route	Page	Access	Stories
/	Home	Public	US-3
/register	Register	Public	US-1
/login	Log in	Public	US-2
/services	Service list with filters	Public	US-3
/services/:id	Service detail with open sessions and spots left	Public to view, login to book	US-4, US-5
/book/:sessionId	Booking form with optional AI instructor note	Logged in	US-6, US-7, US-12
/guide	AI session guide (describe goals, get a recommendation)	Logged in	US-11, US-13
/dashboard	Upcoming sessions, Zoom links, AI summaries	Logged in	US-8, US-10
/bookings	Manage bookings (reschedule, cancel)	Logged in	US-8, US-9
/practice	Home practice, ratings, and progress	Logged in	US-15, US-16, US-17
/instructor	Upcoming bookings with student notes	Instructor only	US-14

Unknown routes show a simple 404 page.

Shared components

- **Layout:** Navbar, Footer, ProtectedRoute (redirects to login), DisclaimerBanner
- **Services:** ServiceCard, ServiceFilters, SlotPicker, SpotsBadge
- **Bookings:** BookingForm, BookingCard, CancelDialog
- **AI:** GuideChat or GuideForm, RecommendationCard, InstructorNote, PracticeCard, RatingForm
- **Feedback states:** LoadingSpinner, EmptyState, ErrorMessage

Auth state lives in an AuthContext, and every API call goes through a small api service so pages never build requests themselves.

Folder structure

```
project/
  client/                  React frontend
    src/
      pages/               one file per route
      components/          reusable UI pieces
      context/             AuthContext
      services/            API calls to the backend
      hooks/               shared logic (for example useBookings)
```

```
  server/                   Node and Express backend
    src/
      routes/               URL paths mapped to controllers
      controllers/          read the request, send the response
      services/             business logic (bookingService, aiService)
      models/               Mongoose schemas
      middleware/           auth, validation, rate limiting, error handler
      config/               database connection and environment variables
  README.md
```

A request flows from a route to a controller to a service to a model. Keeping the AI prompts and the booking rules in the services layer means controllers stay thin and the logic is easy to test.

As-Built Summary

Area	Planned	As built
AI feature	Recommendation, instructor note, home practices, feedback loop	Recommendation and instructor note only, grounded in the real service catalog (Gemini API)
AI safety	Disclaimer and medical boundary	Distress keyword check before any AI call, fixed safety messages (including the 988 line), validated JSON output, 20-second timeout, rate limit of 10 requests per 15 minutes
Instructor model	Name from linked user	Instructor has its own name; the user link is optional
Service type values	group, one-on-one	group, one-on-one (stored as `one-on-one`)
Booking changes	Create and cancel	Create, view, reschedule, and cancel, with an atomic capacity check so a full class cannot be overbooked
Instructor dashboard, email confirmations, practice and feedback collections	Stretch goals	Not built; listed as future work

Testing Summary

Testing combined manual walkthroughs of every user flow with direct API checks. Automated unit tests were not written, which is listed as a limitation.

- **API checks (curl):** register, login, protected routes rejected without a token, booking creation, duplicate booking blocked, full class blocked, cancellation freeing a spot, a user unable to see or cancel another user's booking, and the AI endpoint with valid, too-short, and distress-style input.
- **Manual walkthroughs:** register, log in, log out, browse and filter services, open a detail page, book, reschedule, cancel, and view the dashboard, repeated on the local build and on the deployed site.

- **Issues found and fixed during testing:** the logo logged the user out, the dashboard did not show bookings, and the home button sent logged-in users to the sign-up page. All three were fixed and retested.

AI Feature Explanation

The AI session guide takes a short description of the user's experience and goals (10 to 1,000 characters). The server loads the real service list from MongoDB, sends it with the description to the Gemini API, and requires a JSON answer naming one existing service, a reason, and a short note for the instructor. The server validates that the service exists before showing it. If the text mentions an injury, a medical condition, or distress, the server skips the AI and returns a fixed message to see a doctor or contact support. If the call fails or times out, the user sees a friendly message and can still book without the guide. The recommendation and instructor note are saved on the booking so the dashboard shows them without another AI call.

How AI Was Used in Development

Claude was used as a coding and planning assistant throughout the project:

- **Planning:** comparing project ideas, choosing a feasible theme, and drafting the project idea, problem statement, user stories, and requirements.
- **Design:** proposing the MongoDB schema, the three-tier folder structure, and the booking capacity logic.
- **Implementation:** generating starter code for the Express API, React pages, and the Gemini integration, which was then run, tested, and adjusted.
- **Troubleshooting:** diagnosing errors such as the port conflict on macOS, file-name casing differences between macOS and Linux hosting, missing imports, a package.json typo that failed the Render deploy, and environment variable setup.
- **Documentation:** drafting the README and this document.

All AI-generated code was reviewed, run locally, and tested before it was committed. API keys and database credentials are kept in environment variables and excluded from the repository.